FacebookEyeTracker: An Open-Source Pipeline for Collecting, Processing, and Visualizing Eye-Tracking Data on Social Media Content

Sebastian Breguel
Pontificia Universidad Católica de Chile
sebabreguel@uc.cl

March 29, 2026

Abstract

We present **FacebookEyeTracker**, an open-source pipeline for collecting, processing, and visualizing eye-tracking data while users browse social media feeds. The system captures binocular gaze data via Tobii Pro hardware at ∼60 Hz, cleans and interpolates tracking gaps, temporally aligns gaze samples with individual posts, and generates Gaussian heatmaps and scanpath plots automatically. The pipeline is fully modular—each stage can run independently—and is publicly available under CC BY-NC 4.0.

Keywords: eye tracking, heatmap, scanpath, fixation detection, social media, HCI

1 Introduction

Understanding visual attention on social media feeds is a key question in HCI and UX research [Duchowski, 2017]. While eye tracking provides objective gaze records, synchronizing hardware data with dynamically scrolling web content and producing publication-ready visualizations requires substantial bespoke tooling.

We present **FacebookEyeTracker**, a four-stage pipeline:

1. **Collection** — binocular gaze recording with concurrent screenshot capture.
2. **Processing** — binocular averaging, monocular fallback, and linear interpolation of gaps.
3. **Matching** — temporal alignment of gaze samples with individual posts via server metadata.
4. **Visualization** — Gaussian heatmaps and scanpath plots per post.

Each stage reads from the previous stage's output, supporting both full-pipeline execution and step-by-step debugging. Source code: `https://github.com/sebastianbreguel/FacebookEyeTracker`.

2 Related Work

Duchowski [2017] and Holmqvist et al. [2011] provide the foundational framework for gaze-based studies. PyGaze [Dalmaijer et al., 2014] offers a Python interface to eye trackers but targets controlled experiments rather than naturalistic browsing. Gaussian heatmaps for gaze data follow Wooding [2002], and scanpath visualization builds on Noton and Stark [1971].

Our contribution is coupling these methods with a temporal matching mechanism that links gaze data to individual posts in a scrollable feed—where traditional static AOI definitions are insufficient.

3 System Architecture

Figure 1 shows the pipeline. Each stage is an independent Python script communicating through CSV and JSON files, allowing any component to be replaced without modifying the rest.

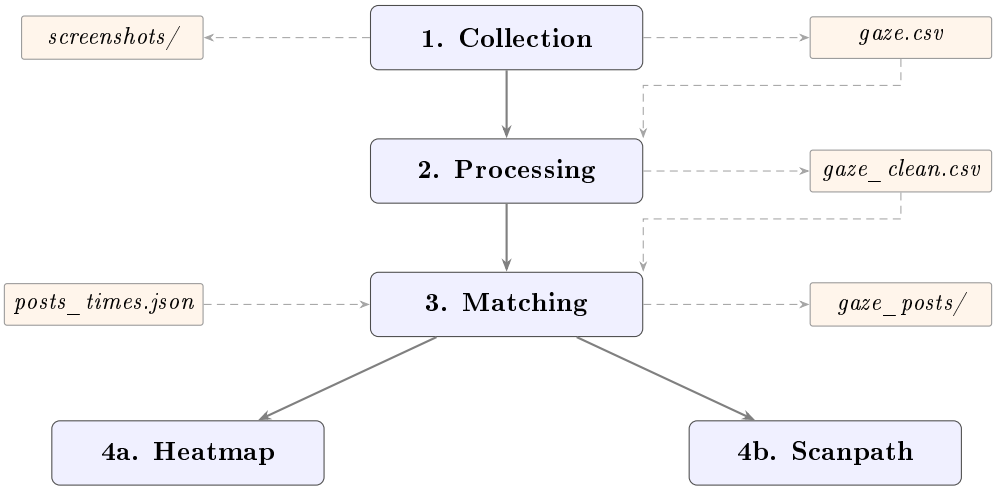

Figure 1: Pipeline architecture. Solid arrows show stage sequence; dashed arrows show data flow.

The system requires a Tobii Pro eye tracker, the Tobii Pro Eye Tracker Manager for calibration, and a backend server on `localhost:3001` providing post metadata.

4 Data Collection

Gaze data is collected via the Tobii Pro SDK using an asynchronous callback. Each sample provides normalized display coordinates $(l_x, l_y, r_x, r_y) \in [0, 1]$ for both eyes, with microsecond SDK timestamps and ISO 8601 wall-clock times. Concurrently, screenshots are captured via PyAutoGUI with timestamped filenames that later serve as metadata for the matching stage.

5 Gaze Data Processing

Binocular averaging. Left and right eye coordinates are averaged into a single gaze point: $\bar{x} = (l_x + r_x)/2$. When one eye is missing, the system falls back to the available eye.

Coordinate normalization. Normalized $[0, 1]$ coordinates are mapped to pixel space by multiplying by display dimensions (1920×1080 by default).

Gap interpolation. Contiguous NaN sequences are filled via linear interpolation between the last valid and next valid samples, preserving smooth trajectories during brief tracking losses.

6 Gaze-to-Post Matching

A backend server provides JSON metadata per post, including start/end times relative to the recording origin. Each gaze sample is assigned to the post whose time window contains its timestamp; samples outside all windows are discarded. Screenshots are matched using the same temporal criterion via their filename timestamps.

7 Visualization

7.1 Gaussian Heatmaps

Gaze density is estimated by accumulating a 2D Gaussian kernel (200×200 px, $\sigma = g/6 \approx 33$ px) at each gaze point. Kernels at display edges are clipped rather than discarded. Values below the non-zero mean are masked, and the result is rendered with the *jet* colormap at $\alpha = 0.5$ over the post screenshot (Figure 2a).

7.2 Scanpath Plots

Fixations are detected with a dispersion-based method: consecutive gaze points within a 450 px radius are grouped into a single fixation. When the distance exceeds this threshold, a new fixation begins. Fixation circles are sized proportionally to $\log(d_f)/\log(d_{\max})$, connected by saccade lines, and labeled with sequential indices (Figure 2b).

(a) Gaussian heatmap

(b) Scanpath plot

Figure 2: Example visualizations for a single post. (a) Heatmap: warm colors indicate high fixation density. (b) Scanpath: circle size $\propto$ log-duration; numbers show viewing order.

8 Pipeline Parameters

Table 1 summarizes default parameters. All are configurable via command-line arguments.

Table 1: Default pipeline parameters.

Stage	**Parameter**	**Default**	**Description**
Collection	Sampling rate	$\sim$60 Hz	Device-dependent
Collection	Display size	1920×1080	Pixels
Processing	Interpolation	Linear	Gap filling
Heatmap	Kernel size	200 px	Gaussian width
Heatmap	σ	$\approx$33 px	Kernel spread
Heatmap	Transparency	0.5	Overlay alpha
Scanpath	Fixation radius	450 px	Dispersion threshold
Scanpath	Circle scaling	Logarithmic	Duration mapping

9 Implementation

The pipeline runs on Python 3.10+ with Matplotlib, NumPy, Pandas, PyAutoGUI, and the Tobii Research SDK. Dependencies are managed with `uv` [Astral Software Inc., 2024]; code quality with Ruff, ty, and Bandit. A batch utility processes multiple participants in parallel.

10 Future Work

Planned extensions include: velocity-based fixation detection (I-VT) [Salvucci and Goldberg, 2000], automatic AOI extraction via computer vision, advanced smoothing (Savitzky–Golay), real-time visualization during recording, and cross-platform support.

11 Conclusion

FacebookEyeTracker provides a modular, open-source solution for eye-tracking studies on social media. Its automatic gaze-to-post matching and dual visualization outputs make it a practical tool for HCI researchers. Code available at `https://github.com/sebastianbreguel/FacebookEyeTracker`.

References

Astral Software Inc. uv: An extremely fast Python package installer and resolver, 2024. URL `https://github.com/astral-sh/uv`. Accessed: 2024.

Edwin S. Dalmaijer, Sebastiaan Mathôt, and Stefan Van der Stigchel. PyGaze: An open-source, cross-platform toolbox for minimal-effort programming of eyetracking experiments. *Behavior Research Methods*, 46(4):913–921, 2014. doi: 10.3758/s13428-013-0422-2.

Andrew T. Duchowski. *Eye Tracking Methodology: Theory and Practice*. Springer, 3rd edition, 2017. doi: 10.1007/978-3-319-57883-5.

Kenneth Holmqvist, Marcus Nyström, Richard Andersson, Richard Dewhurst, Halszka Jarodzka, and Joost van de Weijer. *Eye Tracking: A Comprehensive Guide to Methods and Measures*. Oxford University Press, 2011.

David Noton and Lawrence Stark. Scanpaths in eye movements during pattern perception. *Science*, 171(3968):308–311, 1971. doi: 10.1126/science.171.3968.308.

Dario D. Salvucci and Joseph H. Goldberg. Identifying fixations and saccades in eye-tracking protocols. In *Proceedings of the 2000 Symposium on Eye Tracking Research and Applications (ETRA)*, pages 71–78, 2000. doi: 10.1145/355017.355028.

David S. Wooding. Fixation maps: Quantifying eye-movement traces. *Eye Tracking Research and Applications Symposium (ETRA)*, pages 31–36, 2002. doi: 10.1145/507072.507078.